

TFM Churn E-commerce — Bitácora de avance (Parte 3)

Continuación de TFM Churn E-commerce – Bitácora de avance (Parte 2) . Registra el bloque de **modelado**: pipeline de preprocesamiento, métricas de evaluación y los cuatro modelos del orden de comparación (Dummy → Logistic → Random Forest → XGBoost).

17. Preprocesamiento y métricas — `src/models/baseline.py` , `src/evaluation/metrics.py`

Features numéricas del dataset cliente+snapshot (`NUMERIC_FEATURES` , definidas en `baseline.py` y reutilizadas por todos los modelos):

```
recency, frequency, monetary, n_countries, n_products,
avg_days_between_purchases, month_sin, month_cos, is_post_holiday
```

Excluidas explícitamente: `Customer ID` , `cutoff_date` (no predictoras) y `month` cruda (redundante con `month_sin / month_cos` ; como entero 1-12 induce una escala falsa donde diciembre y enero quedan “lejos”).

Preprocesamiento: `ColumnTransformer` con `SimpleImputer(strategy="median")` + `StandardScaler` (este último solo para modelos lineales; los basados en árboles usan solo imputación, ver más abajo).

Métricas: `evaluate_binary(y_true, proba, threshold=0.5)` en `src/evaluation/metrics.py` — PR-AUC, ROC-AUC, precision, recall, F1.

Cobertura de tests: `tests/test_baseline.py` , `tests/test_metrics.py` .

18. Modelo 1 — DummyClassifier (suelo de referencia)

Estrategia: `strategy="stratified"` — respeta la proporción de clases de train sin usar señal real. Da un suelo probabilístico (ROC-AUC $\approx$ 0,50, PR-AUC $\approx$ prevalencia del split), no solo una heurística de “acertar más que al azar”.

Resultado (split Opción A: train hasta 2010-12, valid ene-abr 2011, test may-sep 2011)

Split	PR-AUC	ROC-AUC	Precision	Recall
train	0,4965	0,5062	0,4997	0,4991
valid	0,6448	0,4995	0,6446	0,4881
test	0,5871	0,5002	0,5871	0,4926

Validación: ROC-AUC $\approx$ 0,50 en los tres splits (sin capacidad discriminativa, como debe ser) y PR-AUC coincide con la prevalencia real de cada split — confirma que `evaluate_binary` está bien implementado.

19. Modelo 2 — Regresión Logística — `src/models/logistic.py`

Configuración: `LogisticRegression(max_iter=2000, class_weight="balanced")` sobre el pipeline con escalado.

Script de entrenamiento: `scripts/train_baseline_logistic.py` (posteriormente ampliado a `scripts/train_all_models.py` para incluir los modelos siguientes).

Resultado

Split	PR-AUC	ROC-AUC	Precision	Recall
train	0,6909	0,7226	0,6351	0,7344
valid	0,8271	0,7735	0,8114	0,7238

test	0,7852	0,7702	0,7755	0,6898
------	--------	--------	--------	--------

Lectura: salto claro respecto al baseline (ROC-AUC 0,50 → 0,72-0,77). `valid / test` rinden igual o mejor que `train` — buena señal de generalización, sin overfitting. Recall/precision equilibrados gracias a `class_weight="balanced"`.

Incidencia resuelta: `ModuleNotFoundError: No module named 'src'`

Al ejecutar `scripts/train_baseline_logistic.py` directamente, Python añade al `sys.path` la carpeta del script (`scripts/`), no la raíz del proyecto — el `pythonpath = ["."]` de `pyproject.toml` solo cubre `pytest`, no `scripts` sueltos.

Solución adoptada (patrón a repetir en todo script nuevo de `scripts/`):

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
```

20. Modelo 3 — Random Forest — `src/models/random_forest.py`

Primer intento (sin regularizar, `n_estimators=300`, sin límite de profundidad):

Split	PR-AUC	ROC-AUC	Precision	Recall
train	1,0000	1,0000	0,9999	1,0000
valid	0,8224	0,7554	0,8285	0,5216
test	0,7827	0,7522	0,7841	0,4915

`train = 1,0000` en todas las métricas es sobreajuste severo — el modelo memoriza `train` en vez de generalizar. No es una comparación válida contra la Logística en este estado.

Regularización aplicada: `max_depth=6`, `min_samples_leaf=50` (con 32.537 filas en `train`, un mínimo razonable, no agresivo), manteniendo `class_weight="balanced"`.

Resultado regularizado

Split	PR-AUC	ROC-AUC	Precision	Recall
train	0,7499	0,7720	0,6804	0,7322
valid	0,8357	0,7735	0,8470	0,4735
test	0,8104	0,7789	0,8005	0,5246

Lectura: tras regularizar, `train/valid/test` quedan en rango similar (ROC-AUC ~0,77-0,78) — ya no hay overfitting evidente. En métricas independientes del threshold (PR-AUC, ROC-AUC), el RF va ligeramente por delante de la Logística. Pero **a `threshold=0,5` el recall cae fuerte** (0,47-0,52 frente a 0,69-0,72 de la Logística) — la función de decisión del RF está calibrada de forma distinta, y comparar recall a un threshold fijo de 0,5 entre dos modelos con distribuciones de probabilidad distintas no es una comparación justa.

Conclusión provisional (no definitiva): no se puede declarar un ganador real entre Logística y RF hasta hacer calibración + selección de threshold (sección 21, pendiente).

21. Modelo 4 — XGBoost — `src/models/xgboost_model.py`

Configuración: valores conservadores desde el inicio (no el default de XGBoost, más agresivo), dado lo observado con Random Forest sin regularizar — `max_depth=4`, `learning_rate=0.05`, `n_estimators=300`.

Tratamiento del desbalance: XGBoost no soporta `class_weight="balanced"` de scikit-learn; se usa el equivalente `scale_pos_weight` (proporción negativos/positivos en `train`), calculado con `compute_scale_pos_weight(y)`.

Cobertura de tests: `tests/test_xgboost_model.py`, incluyendo verificación del cálculo de `scale_pos_weight`.

Estado: código implementado e integrado en `scripts/train_all_models.py` junto a los tres modelos anteriores.

Ejecución sobre datos reales pendiente — resultados aún no disponibles a la hora de cerrar esta bitácora.

22. Estado actual y próximos pasos

✓ Completado en esta sesión (bloque de modelado)

- ☒ `src/models/baseline.py` — preprocesamiento (`ColumnTransformer`) + `DummyClassifier` como suelo de referencia.
- ☒ `src/evaluation/metrics.py` — PR-AUC, ROC-AUC, precision, recall, F1.
- ☒ `src/models/logistic.py` — Regresión Logística con `class_weight="balanced"`, supera claramente al baseline.
- ☒ `src/models/random_forest.py` — Random Forest, primer intento sobreajustado detectado y corregido con regularización (`max_depth=6`, `min_samples_leaf=50`).
- ☒ `src/models/xgboost_model.py` — pipeline implementado y testeado unitariamente; integrado en el script de comparación.
- ☒ Incidencia de `sys.path` en scripts sueltos resuelta y documentada como patrón a repetir.

» Próximos pasos

- ☐ **Ejecutar** `scripts/train_all_models.py` con los cuatro modelos (dummy, logistic, random_forest, xgboost) y registrar los resultados de XGBoost, pendientes en el momento de cerrar esta bitácora.
 - ☐ **Calibración + selección de threshold** (sección 12 del doc técnico) — necesario para comparar de forma justa Logística vs Random Forest vs XGBoost más allá del threshold por defecto (0,5), dado que ya se detectó que el RF cambia mucho de recall/precision según el punto de corte.
 - ☐ Decidir modelo candidato final en base a la comparación calibrada, no solo a PR-AUC/ROC-AUC.
 - ☐ `src/explainability/` — SHAP sobre el modelo candidato final.
 - ☐ **Decidir tratamiento de clientes mayoristas** (`is_wholesale`, 70 clientes, 1,19 %) — sigue pendiente desde la Parte 1, aún no abordado.
 - ☐ Incorporar MLflow para trazabilidad de experimentos.
 - ☐ `src/api/` — servicio FastAPI de inferencia.
-

Última actualización: registro generado tras implementar los cuatro modelos del orden de comparación y detectar/corregir overfitting en Random Forest — a la espera de ejecutar XGBoost sobre datos reales y de la calibración/threshold tuning antes de seleccionar el modelo final.